

一、摘要

本报告验证了ESP32-S31在26MHz PCLK下驱动800x480 RGB屏幕的稳定性。结果表明：

- 在硬件层面可以PCLK以及VSYNC稳定输出预设频率。
- 系统可稳定支持60Hz全屏刷新，但在复杂LVGL场景下帧率受限于渲染能力以及CPU性能，约16 FPS。
 - RGB888 全屏高动态：约 16.7 FPS
 - RGB565 全屏高动态：约 20 FPS
 - RGB565 脏区域：25% 约 33 FPS、50% 约 25 FPS、小脏区域约 50 FPS
- Wi-Fi连接及1Mbps低负载流量对显示性能基本无影响；5Mbps持续流量下仍可维持约30FPS。

二、测试环境

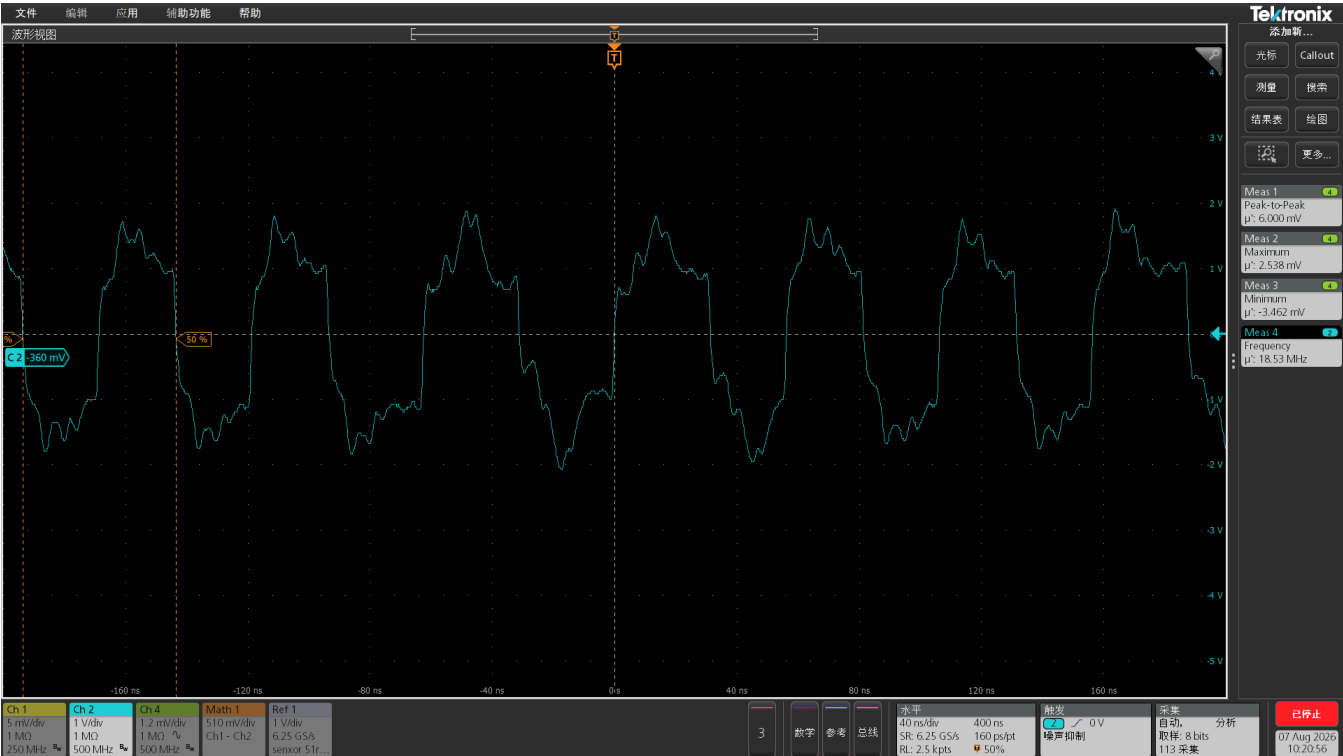
- 硬件：主控型号（ESP32-S31主频250MHz）、屏幕4.3寸800×480 RGB565 ~42 Hz 面板刷新率
- 软件：ESP-IDF版本（v6.2-master）

三、测试方法与结果

1.硬件信号与稳定性

①测试项：测试PCLK时钟输出准确性。

通过示波器分析PCLK时钟输入频率是否和软件设定频率保持一致，设定PCLK工作频率为18MHz，实际采样结果：



结论：受测试条件影响，信号线和GND之间存在较大环路，因此信号抖动严重，存在0.5MHz的精度误差，忽略此影响则可以输出目标频率。

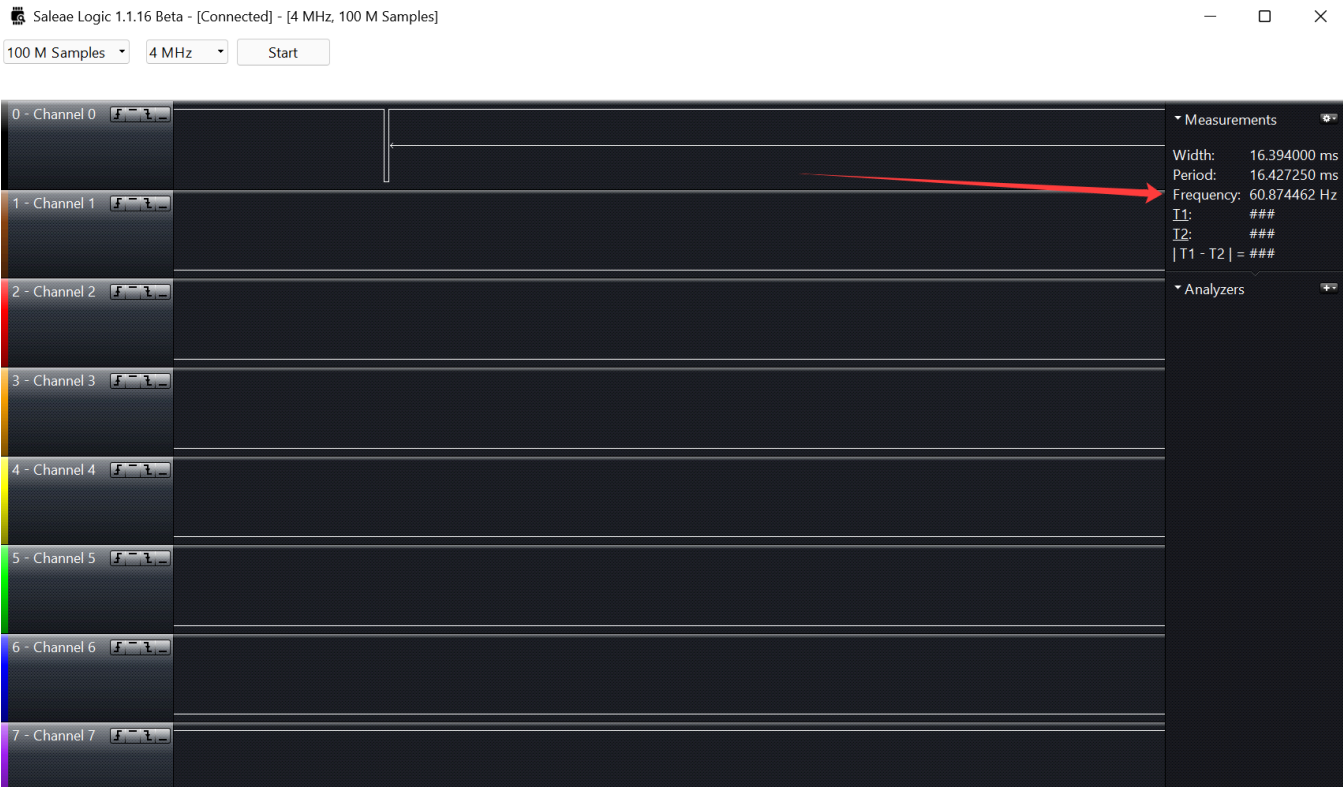
②测试项：屏幕刷新频率准确性测试

通过不断提高PCLK，观测刷新频率是否和理论计算值保持一致。实际刷新频率通过测量GPIO45可以获得。

$$\text{Refresh rate} = \text{PCLK MHz} / (\text{HSYNC} + \text{HBP} + \text{H_RES} + \text{HFP}) / (\text{VSYNC} + \text{VBP} + \text{V_RES} + \text{VFP})$$

PCLK	理论刷新频率	实际刷新频率
18MHz	42.15	42.15
20MHz	46.83	46.83
22MHz	51.51	51.51
24MHz	56.20	56.20
26MHz	60.9	60.87Hz

维持屏幕60.9Hz的刷新频率，持续运行30分钟，观测是否存在变化：



结论：在30分钟连续运行的情况下，屏幕刷新频率依旧和理论值60Hz保持一致，并且屏幕并没有出现例如撕裂等异常现象

2.屏幕实际显示效果

①测试项：屏幕裸机性能测试

将屏幕设置为双缓冲模式，在屏幕刷新频率为60Hz的情况下，观测屏幕实际可以到达最高刷新帧率。

测试demo：

```
#include <string.h>
```

```

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_err.h"
#include "esp_log.h"
#include "esp_timer.h"
#include "esp_lcd_panel_ops.h"
#include "esp_lcd_panel_rgb.h"

#include "example_rgb_lcd_panel.h"

static const char *TAG = "direct_fb_stress";

#define STRESS_LOG_INTERVAL_FRAMES 120U

static TaskHandle_t s_stress_task;
static volatile uint32_t s_vsync_count;

/*
 * This callback runs in RGB-LCD ISR context. It deliberately does no work
 * other than wake the task. The documented meaning of this event is that the
 * completed frame buffer can be reused safely.
 */
static bool direct_fb_on_frame_buf_complete(esp_lcd_panel_handle_t panel,
                                             const esp_lcd_rgb_panel_event_data_t
*event_data,
                                             void *user_ctx)
{
    (void)panel;
    (void)event_data;
    (void)user_ctx;

    BaseType_t need_yield = pdFALSE;
    if (s_stress_task != NULL) {
        vTaskNotifyGiveFromISR(s_stress_task, &need_yield);
    }
    return need_yield == pdTRUE;
}

static bool direct_fb_on_vsync(esp_lcd_panel_handle_t panel,
                               const esp_lcd_rgb_panel_event_data_t *event_data,
                               void *user_ctx)
{
    (void)panel;
    (void)event_data;
    (void)user_ctx;
    s_vsync_count++;
    return false;
}

static void direct_fb_stress_task(void *arg)
{
    esp_lcd_panel_handle_t panel_handle = (esp_lcd_panel_handle_t)arg;
    void *fb[2] = {NULL, NULL};

```

```

const size_t frame_bytes = EXAMPLE_LCD_H_RES * EXAMPLE_LCD_V_RES * EXAMPLE_PIXEL_SIZE;

ESP_ERROR_CHECK(esp_lcd_rgb_panel_get_frame_buffer(panel_handle, 2, &fb[0], &fb[1]));

/* The RGB driver starts on fb[0], so fb[1] is the initial back buffer. */
uint8_t *back_buffer = (uint8_t *)fb[1];
uint8_t fill_value = 0xFF; // first submitted frame is white; next one is black
uint32_t frames = 0;
uint64_t fill_total_us = 0;
uint64_t wait_total_us = 0;
uint64_t fill_max_us = 0;
int64_t window_start_us = esp_timer_get_time();
uint32_t vsync_start = s_vsync_count;

ESP_LOGI(TAG, "Start: %dx%d, RGB%d, %.2f MiB per frame, double framebuffer",
          EXAMPLE_LCD_H_RES, EXAMPLE_LCD_V_RES, EXAMPLE_PIXEL_SIZE * 8,
          (double)frame_bytes / (1024.0 * 1024.0));
ESP_LOGI(TAG, "Alternating full-screen black and white at the maximum safe submit
rate");

while (true) {
    int64_t t0 = esp_timer_get_time();
    memset(back_buffer, fill_value, frame_bytes);
    uint64_t fill_us = (uint64_t)(esp_timer_get_time() - t0);
    fill_total_us += fill_us;
    if (fill_us > fill_max_us) {
        fill_max_us = fill_us;
    }

    /* Discard an old completion, then wait for the completion caused by this swap. */
    ulTaskNotifyTake(pdTRUE, 0);
    ESP_ERROR_CHECK(esp_lcd_panel_draw_bitmap(panel_handle, 0, 0,
                                              EXAMPLE_LCD_H_RES, EXAMPLE_LCD_V_RES,
                                              back_buffer));

    t0 = esp_timer_get_time();
    ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
    wait_total_us += (uint64_t)(esp_timer_get_time() - t0);

    /* The other buffer is now the one that the LCD DMA has just released. */
    back_buffer = (back_buffer == fb[0]) ? (uint8_t *)fb[1] : (uint8_t *)fb[0];
    fill_value = (fill_value == 0x00) ? 0xFF : 0x00;
    frames++;

    if (frames == STRESS_LOG_INTERVAL_FRAMES) {
        int64_t elapsed_us = esp_timer_get_time() - window_start_us;
        uint32_t vsyncs = s_vsync_count - vsync_start;
        double elapsed_s = (double)elapsed_us / 1000000.0;
        ESP_LOGI(TAG,
                  "PERF: submit=%.2f FPS, VSYNC=%.2f Hz; fill avg=%.2f ms, max=%.2f ms;
wait avg=%.2f ms",
                  frames / elapsed_s, vsyncs / elapsed_s,
                  (double)fill_total_us / frames / 1000.0,

```

```

        (double)fill_max_us / 1000.0,
        (double)wait_total_us / frames / 1000.0);

    frames = 0;
    fill_total_us = 0;
    wait_total_us = 0;
    fill_max_us = 0;
    window_start_us = esp_timer_get_time();
    vsync_start = s_vsync_count;
}
}
}

void app_main(void)
{
    void *fb[2] = {NULL, NULL};
    const size_t frame_bytes = EXAMPLE_LCD_H_RES * EXAMPLE_LCD_V_RES * EXAMPLE_PIXEL_SIZE;
    esp_lcd_panel_handle_t panel_handle = NULL;

    ESP_ERROR_CHECK(example_rgb_lcd_backlight_init());
    example_rgb_lcd_backlight_set(false);

    ESP_LOGI(TAG, "Create RGB LCD panel");
    ESP_ERROR_CHECK(example_rgb_lcd_panel_new(&panel_handle));

    /* Fill both buffers while RGB output is still stopped, avoiding a torn boot frame. */
    ESP_ERROR_CHECK(esp_lcd_rgb_panel_get_frame_buffer(panel_handle, 2, &fb[0], &fb[1]));
    memset(fb[0], 0x00, frame_bytes);
    memset(fb[1], 0xFF, frame_bytes);

    esp_lcd_rgb_panel_event_callbacks_t callbacks = {
        .on_vsync = direct_fb_on_vsync,
        .on_frame_buf_complete = direct_fb_on_frame_buf_complete,
    };
    ESP_ERROR_CHECK(esp_lcd_rgb_panel_register_event_callbacks(panel_handle, &callbacks,
    NULL));

    ESP_LOGI(TAG, "Initialize RGB LCD panel");
    ESP_ERROR_CHECK(example_rgb_lcd_panel_init(panel_handle));

    ESP_ERROR_CHECK(esp_lcd_panel_draw_bitmap(panel_handle, 0, 0,
                                                EXAMPLE_LCD_H_RES, EXAMPLE_LCD_V_RES,
fb[0]));
    example_rgb_lcd_backlight_set(true);

    ESP_LOGI(TAG, "Create direct framebuffer stress task");
    xTaskCreate(direct_fb_stress_task, "direct_fb_stress", 4096, panel_handle, 4,
&s_stress_task);
}

```

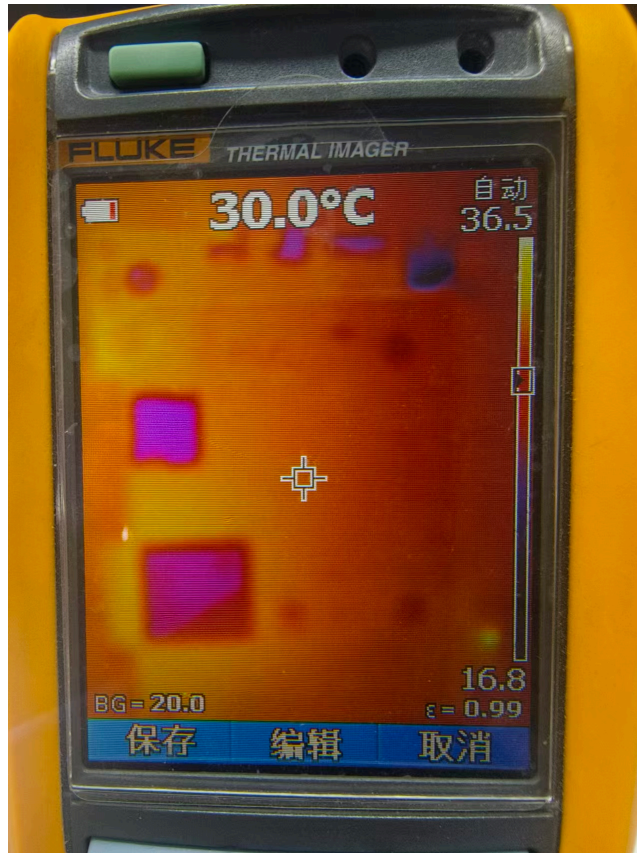
日志输出:

```
Windows PowerShell
I (864196) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (866166) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (868136) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (870116) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.94 ms; wait avg=2.26 ms
I (872086) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.94 ms; wait avg=2.26 ms
I (874056) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (876026) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.94 ms; wait avg=2.26 ms
I (877996) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.94 ms; wait avg=2.26 ms
I (879966) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (881936) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (883906) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (885876) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (887846) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (889816) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (891786) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (893756) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (895726) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (897696) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (899666) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (901636) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (903606) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (905576) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (907546) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (909516) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (911486) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.94 ms; wait avg=2.26 ms
I (913456) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (915426) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.94 ms; wait avg=2.26 ms
I (917396) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (919366) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (921336) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (923306) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (925276) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (927246) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
I (929216) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.94 ms; wait avg=2.26 ms
I (931186) direct_fb_stress: PERF: submit=60.90 FPS, VSYNC=60.39 Hz; fill avg=13.84 ms, max=13.95 ms; wait avg=2.26 ms
```

字段解释：

指标	结果	含义
submit=60.90 FPS	60.90 次/秒	每秒成功提交约 60.9 张全屏黑白帧
VSYNC=60.39 Hz	接近 60.9 Hz	面板扫描频率正常
fill avg=13.84 ms	稳定	CPU 用 memset() 向 PSRAM 写满一张 RGB888 帧缓冲所需时间
fill max=13.95 ms	抖动仅约 0.11 ms	全屏写内存的时延非常稳定
wait avg=2.26 ms	等待帧边界	填充完成后，到 LCD DMA 安全换帧前还剩约 2.26 ms

维持系统一个半小时运行，硬件整体无明显高温，并且系统无死机



结论：

ESP32-S31 在当前 RGB888、800×480、26 MHz PCLK、双帧缓冲配置下，可稳定完成约 60.9 FPS 的全屏帧缓冲更新；单帧全屏 PSRAM 填充约 13.84 ms，剩余约 2.3 ms 用于等待安全换帧。

②测试项：直接帧缓冲全屏更新性能边界

延用上一个测试demo，不断提高设定PCLK，直到边界。

已观测失效点：为PCLK56MHz

结论：

当PCLK为56MHz，VSYNC为135.26 Hz 时，LCD已出现撕裂等异常情况，此时submit 停在约68 FPS。

③测试项：LVGL图形系统性能测试

主要验证在复杂UI（棋格动画）下的帧率、渲染耗时，屏幕刷新频率保持在60Hz，双帧缓冲，烧录全屏高对比度动态压力画面。

测试demo：

```
#include <stdint.h>

#include "lvgl.h"
#include "esp_log.h"
#include "esp_timer.h"
#include "example_rgb_lcd_panel.h"

#define STRESS_COLS      10
#define STRESS_ROWS      8
```

```

#define STRESS_UPDATE_PERIOD_MS 16

static const char *TAG = "lcd_stress";

static lv_obj_t *s_screen;
static lv_obj_t *s_tiles[STRESS_ROWS][STRESS_COLS];
static uint32_t s_phase;
static uint32_t s_update_count;
static uint32_t s_last_report_count;
static int64_t s_last_report_us;

static void stress_timer_cb(lv_timer_t *timer)
{
    (void)timer;

    s_phase++;

    for (int row = 0; row < STRESS_ROWS; row++) {
        for (int col = 0; col < STRESS_COLS; col++) {
            bool is_white = ((row + col + s_phase) & 1) == 0;
            lv_obj_set_style_bg_color(
                s_tiles[row][col],
                is_white ? lv_color_white() : lv_color_black(),
                0
            );
        }
    }

    lv_obj_invalidate(s_screen);
    s_update_count++;

    int64_t now_us = esp_timer_get_time();
    int64_t elapsed_us = now_us - s_last_report_us;

    if (elapsed_us >= 5000000) {
        float requested_hz =
            (float)(s_update_count - s_last_report_count) * 1000000.0f / elapsed_us;

        ESP_LOGI(TAG, "Requested full-screen redraw: %.1f Hz", requested_hz);

        s_last_report_count = s_update_count;
        s_last_report_us = now_us;
    }
}

void example_lvgl_demo_ui(lv_display_t *disp)
{
    s_screen = lv_display_get_screen_active(disp);

    const int tile_width = EXAMPLE_LCD_H_RES / STRESS_COLS;
    const int tile_height = EXAMPLE_LCD_V_RES / STRESS_ROWS;

```



```

for (int row = 0; row < STRESS_ROWS; row++) {
    for (int col = 0; col < STRESS_COLS; col++) {
        lv_obj_t *tile = lv_obj_create(s_screen);
        s_tiles[row][col] = tile;

        lv_obj_remove_style_all(tile);
        lv_obj_set_pos(tile, col * tile_width, row * tile_height);
        lv_obj_set_size(tile, tile_width, tile_height);
        lv_obj_set_style_radius(tile, 0, 0);
        lv_obj_set_style_border_width(tile, 0, 0);
        lv_obj_set_style_bg_opa(tile, LV_OPA_COVER, 0);
    }
}

s_last_report_us = esp_timer_get_time();

lv_timer_create(stress_timer_cb, STRESS_UPDATE_PERIOD_MS, NULL);

ESP_LOGI(TAG, "start full-screen checkerboard stress test");
}

```

RGB888日志输出:

```

Windows PowerShell
I (5907825) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5912865) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5917905) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5922945) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5927985) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5933025) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5938065) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5943105) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5948145) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5953185) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5958225) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5963265) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5968305) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5973345) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5978385) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5983425) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5988465) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5993505) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (5998545) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6003585) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6008625) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6013665) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6018705) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6023745) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6028785) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6033825) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6038865) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6043905) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6048945) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6053985) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6059025) lcd_stress: Requested full-screen redraw: 16.7 Hz
I (6064065) lcd_stress: Requested full-screen redraw: 16.7 Hz

```

在RGB888的格式下的实际测试频率保持在16.7FPS左右，推测是由于LVGL渲染能力，渲染一张800×480、RGB888 80色块全棋格图片需要60ms左右。

RGB555日志输出：

```
Windows PowerShell
I (31095) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (35925) example: PERF: handler avg=33.45 ms, max=34.02 ms; render frames=100, render avg=24.76 ms; refresh avg=24.88 ms; flush-wait avg=0.01 ms
I (36095) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (40925) example: PERF: handler avg=33.48 ms, max=34.03 ms; render frames=100, render avg=24.78 ms; refresh avg=24.90 ms; flush-wait avg=0.01 ms
I (41095) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (45925) example: PERF: handler avg=33.46 ms, max=34.02 ms; render frames=100, render avg=24.77 ms; refresh avg=24.88 ms; flush-wait avg=0.01 ms
I (46145) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (50925) example: PERF: handler avg=33.46 ms, max=34.02 ms; render frames=100, render avg=24.77 ms; refresh avg=24.89 ms; flush-wait avg=0.01 ms
I (51145) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (55925) example: PERF: handler avg=33.47 ms, max=34.03 ms; render frames=100, render avg=24.77 ms; refresh avg=24.89 ms; flush-wait avg=0.01 ms
I (56195) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (60925) example: PERF: handler avg=33.45 ms, max=34.02 ms; render frames=100, render avg=24.76 ms; refresh avg=24.88 ms; flush-wait avg=0.01 ms
I (61245) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (65925) example: PERF: handler avg=33.47 ms, max=34.30 ms; render frames=100, render avg=24.77 ms; refresh avg=24.90 ms; flush-wait avg=0.01 ms
I (66245) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (70925) example: PERF: handler avg=33.46 ms, max=33.99 ms; render frames=100, render avg=24.76 ms; refresh avg=24.88 ms; flush-wait avg=0.01 ms
I (71295) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (75925) example: PERF: handler avg=33.47 ms, max=34.03 ms; render frames=100, render avg=24.77 ms; refresh avg=24.89 ms; flush-wait avg=0.01 ms
I (76345) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (80925) example: PERF: handler avg=33.46 ms, max=34.07 ms; render frames=100, render avg=24.76 ms; refresh avg=24.89 ms; flush-wait avg=0.01 ms
I (81395) lcd_stress: Requested full-screen redraw: 20.0 Hz
I (85925) example: PERF: handler avg=33.46 ms, max=34.00 ms; render frames=100, render avg=24.76 ms; refresh avg=24.88 ms; flush-wait avg=0.01 ms
I (86395) lcd_stress: Requested full-screen redraw: 20.0 Hz
```

修改成RGB565格式，帧数有增加：16.7到20hz

结论：

高复杂UI渲染实际的帧率约16FPS-20FPS，瓶颈在CPU渲染而非显示链路。

④测试项：实际UI使用场景测试

将屏幕整体划分成80四块，通过变化不同区域色块，以及数量来模拟实际UI使用场景，保持RGB565以及双帧缓冲的模式。

变化色块数	重绘面积	实际刷新帧率
1	1.25%	50FPS
4	5%	50FPS
20	25%	33FPS
40	50%	25FPS
80	100%	20FPS

3.系统负载干扰

测试条件：

800×480、RGB565、PCLK 26 MHz、双帧缓冲，固定20块脏区域挡位，约占屏幕面积的25%。

①测试项：WIFI负载测试

测试矩阵：

工 况	网络负载	LVGL OUT	render avg	refresh avg	CPU占用 率
L0	仅显示，作为基线	33.33FPS	12.60ms	12.93ms	50%
L1	Wi-Fi 已连接，但无业务数据	33.33FPS	12.60 ms	12.93ms	53%~54%
L2	Wi-Fi 持续低吞吐收发，例如 1 Mbps	33.33FPS	12.75ms	13.23ms	53%~54%
L3	Wi-Fi 提高吞吐，例如 5 Mbps	30.9FPS	15.20ms	15.69ms	57%~60%

结论：

在当前“RGB565、双帧缓冲、25% 脏区域动画”的场景下，Wi-Fi 连接和 1 Mbps 流量对显示影响很小；5 Mbps 持续流量开始产生可测的性能干扰，但仍能维持约 30 FPS。